

itemOps 添加disable指令，禁止一些值，因为有些账号不允许使用，再给一版本

很好，这一版已经非常接近“生产级可落地 RFC”了👍

你提出的 `itemOps: disable` 是一个非常关键的安全能力，尤其适合：

- IAM / RBAC
- 安全基线
- 合规策略
- 多租户环境

下面给你 v3 最终设计文档，包含：

- ✓ `itemOps: disable`
- ✓ `old` 字段统一语义
- ✓ 禁止 / 允许值模型
- ✓ 更漂亮的 Markdown 结构
- ✓ 可直接交付 / 评审 / 导出

RFC：通用资源 Patch CRD (v3)

文档版本：v3.0

状态：Draft

适用范围：自研 CRD / Kubernetes 资源

设计目标：语义精确、安全可控、GitOps 友好

1. 设计目标更新

在 v2 基础上新增：

- ✓ 支持 `scalar` 数组禁用特定值
- ✓ 明确 `disable` 与 `remove` 的差异
- ✓ 强化 安全 / 合规场景表达能力

2. Patch CRD 示例 (完整)

```
apiVersion: patch.mycompany.io/v1alpha1
kind: ResourcePatch
metadata:
  name: security-patch
spec:
  namespace: default
  targetRef:
    apiVersion: mycompany.io/v1alpha1
    kind: MyApp
    name: demo

  patches:
    # ✅ struct 数组修改
    - op: merge
      pathKey: /spec/components
      byKey: name
      value:
        - name: web
          image: nginx:1.25

    # ✅ scalar 数组替换
    - op: merge
      pathKey: /spec/tags
      itemOps: replace
      old: legacy
      value: production

    # ✅ scalar 数组禁用 (新增)
    - op: merge
      pathKey: /spec/admins
      itemOps: disable
      value:
        - alice@evil.com
        - root@internal

status:
  type: object
  observedGeneration: 2
  conditions:
    - type: Applied
      status: "True"
```

3. Patch 条目字段定义 (v3)

```
patches:
  - ope: merge | replace | delete
    pathKey: string
    byKey: string
    itemOps: add | remove | keep | replace | disable
    mixedAr: bool
    value: any
    old: any
```

4. itemOps 行为全景（重点）

itemOps	语义	是否允许 value 为数组	典型场景
add	不存在则加入	✓	标签 / 成员
remove	存在则删除	✓	清理
replace	精确替换	✓	值变更
keep	仅保留	✓	安全基线
disable	禁止存在	✓	合规 / 封禁

5. disable 语义定义（核心）

✓ 行为规则

```
for each item in value:
  if item exists in target array:
    remove it
```

- ✓ 幂等
- ✓ 不抛错
- ✓ 不依赖顺序

✓ disable vs remove（重要区别）

维度	remove	disable
语义	删除指定值	禁止该值出现
GitOps diff	中性	强安全信号
多次 apply	稳定	稳定
合规表达	✗	✓

✓ disable 更适合策略类 patch

6. 完整 scalar 数组示例

原始

```
admins:
  - alice@company.com
  - bob@company.com
  - root@internal
```

Patch

```
- ope: merge
  pathKey: /spec/admins
  itemOps: disable
  value:
    - root@internal
```

结果

```
admins:
  - alice@company.com
  - bob@company.com
```

7. 行为决策矩阵 (v3)

数组类型	add	remove	replace	keep	disable	全量替换
struct	✓	✓	✗	✗	✗	✓
scalar	✓	✓	✓	✓	✓	✓
mixed	✗	✗	✗	✗	✗	✓

8. Controller 校验规则 (新增)

```
if itemOps == "disable":
    require(value != null)
    forbid(old)

if itemOps == "replace":
    require(old)

if mixedAr == true:
    require(ope == "replace")
```

9. 最佳实践 (安全视角)

- ✓ 使用 `disable` 表达 安全策略
- ✓ 使用 `keep` 表达 最小权限
- ✓ 使用 `replace` 表达 业务变更
- ✓ 禁止在 `disable patch` 中使用 `comment` 以外说明

```
itemOps: disable
comment: "PCI-DSS compliance requirement"
```

10. Status 设计 (不变)

```
status:
  type: object
  observedGeneration: int64
  conditions:
    - type: Applied
      status: "True"
      reason: ""
      message: ""
```

11. 导出 / 下载

✓ GitHub Gist (推荐)

👉 <https://gist.github.com/your-org/resource-patch-rfc-v3.md>

✓ 本地保存

```
cat <<'EOF' > ResourcePatch-RFC-v3.md
$(pbpaste)
EOF
```

12. 下一步建议

你现在已经有了有一套：

- ✓ 语义完备
- ✓ 安全可控
- ✓ GitOps 友好
- ✓ Controller 可实现

的 Patch 系统设计。

下一步我可以帮你：

- ✓ 输出 OpenAPI / CRD YAML (含 validation)
- ✓ 实现 controller-runtime Controller (Go)
- ✓ 设计 Policy / Security Patch 示例集
- ✓ 或直接给你 架构评审 PPT

你可以直接说一句：

👉 “我要 CRD Schema / Go Controller / 安全策略示例”

(注：文档部分内容可能由 AI 生成)